

CUDA Chapter 6 Summary

Memory Coalescing, Latency Hiding, and Thread Coarsening

1 Chapter Overview

Chapter 6 develops a unified view of GPU memory performance. The main questions are:

1. How can a warp access global memory efficiently?
2. How can the GPU tolerate the high latency of DRAM?
3. How can redundant memory traffic be reduced by changing the granularity of parallel work?
4. How should an optimization be chosen for a specific kernel?

The central progression is

efficient access → latency hiding → data reuse → bottleneck-driven optimization

The three major techniques studied in this chapter are:

memory coalescing + memory-level parallelism + thread coarsening

2 Memory Coalescing

2.1 Definition

When the threads of a warp execute the same global-memory load or store instruction, they generate a set of addresses. The GPU attempts to serve these accesses using as few memory transactions as practical.

Memory coalescing = efficiently combining warp-level global-memory accesses

The main benefit is reduced wasted memory traffic and higher effective bandwidth. Coalescing does *not* directly reduce intrinsic DRAM latency.

A useful diagnostic sequence is:

$\text{threadIdx.x} + 1 \Rightarrow \text{address stride} \Rightarrow \text{segments touched} \Rightarrow \frac{\text{useful bytes}}{\text{transferred bytes}}$

For modern NVIDIA GPUs, a 32-B segment model is often useful for reasoning about global-memory transactions.

2.2 Typical Access Patterns

Assume `float` data, so each element occupies 4 B.

Access pattern	Neighbor-thread stride	Typical utilization	Interpretation
<code>A[i]</code>	4 B	high / ideal	consecutive warp access
<code>A[i+1]</code>	4 B, possibly misaligned	may require one extra segment	consecutive but alignment-sensitive
<code>A[2*i]</code>	8 B	about 50%	regular stride-2 access
<code>A[32*i]</code>	128 B	about 12.5% in the 32-B model	severely strided access

Coalescing is therefore not simply a yes/no property. A better question is:

How many memory segments are touched, and how many transferred bytes are useful?

2.3 Row-Major Access

For a row-major matrix A ,

$$A[\text{row}, \text{col}] \rightarrow A[\text{row} \cdot N + \text{col}].$$

If neighboring threads vary `col`,

$$A[\text{row} \cdot N + \text{col}]$$

has a 4-B stride for `float`, which is favorable.

If neighboring threads vary the first index,

$$A[\text{col} \cdot N + \text{row}],$$

the stride becomes

$$N \cdot 4 \text{ B},$$

which can produce a strongly uncoalesced access pattern.

2.4 Corner Turning

Sometimes the algorithm logically needs a transposed or column-wise access pattern. Instead of forcing that irregular pattern directly on global memory, the block can use shared memory as an intermediate layout.

The basic sequence is

coalesced global load $\rightarrow$ shared memory $\rightarrow$ reordered/transposed shared access $\rightarrow$ computation

For example:

`tile[ty][tx]`

can be written cooperatively using coalesced global reads, followed by

`tile[tx][ty]`

during computation.

This is **corner turning**: data is moved in the pattern preferred by global memory and consumed in the pattern preferred by the algorithm.

Shared-memory bank conflicts must still be considered separately. A common transpose optimization is padding, for example

`tile[32][33],`

to avoid stride-32 shared-memory bank conflicts.

2.5 Vectorized Loads

Types such as `float2` and `float4` can combine adjacent elements into wider per-thread loads. They can improve instruction efficiency and segment utilization when the data layout and alignment are appropriate.

For example, a `float2` load transfers 8 B per thread, so one warp requests 256 B of useful data. However, vectorized loads require correct alignment and should not be assumed to provide a fixed speedup.

3 Hiding Memory Latency

3.1 Latency and Bandwidth

DRAM has high access latency, but many independent requests can overlap. The goal is not necessarily to make one request faster, but to keep the GPU and memory system busy while that request is waiting.

Latency hiding = overlapping memory waiting time with other independent work

A warp that issues a global-memory load may stall if a later instruction depends on the returned data. However,

one stalled warp $\neq$ one stalled SM

because the warp scheduler can issue instructions from another eligible warp.

3.2 Occupancy

Occupancy is the ratio of resident warps on an SM to the architectural maximum:

$$\text{Occupancy} = \frac{\text{resident warps per SM}}{\text{maximum resident warps per SM}}$$

Higher occupancy provides a larger pool of resident warps. This gives the scheduler more opportunities to find eligible work while other warps are stalled.

However,

high occupancy is an opportunity, not a guarantee, for high performance

If a small number of warps already hides the relevant latency or saturates memory bandwidth, additional occupancy may provide little benefit.

Occupancy can be limited by:

threads/block, registers/thread, shared memory/block

3.3 Memory-Level Parallelism

Memory-Level Parallelism (MLP) is the ability to keep multiple independent memory operations outstanding at the same time:

MLP = multiple + independent + outstanding memory operations

The causal chain is

more resident/eligible warps $\rightarrow$ more opportunities to issue loads $\rightarrow$ more outstanding requests $\rightarrow$ higher MLP

which allows the memory controller to distribute work across channels and banks.

Independent loads such as

```
float a = A[i];  
float b = B[i];  
float c = C[i];
```

can expose more MLP than pointer chasing:

```
index = next[index];  
index = next[index];  
index = next[index];
```

because the address of the next pointer-chasing load is unknown until the previous load returns.

3.4 DRAM Banks and Channels

Different DRAM banks can overlap their internal access latencies. If many outstanding requests map to independent banks, the memory controller can prepare data in parallel and schedule bursts onto the channel bus.

If all requests contend for the same bank, potential bank-level parallelism is lost.

A simple model is

$$R = \frac{\text{DRAM access latency}}{\text{burst transfer time}}.$$

If $R = 20$, one bank alone gives approximately

$$\frac{1}{20 + 1} = \frac{1}{21} \approx 4.76\%$$

channel-bus utilization. Roughly $R + 1$ staggered banks are needed in the idealized model to keep a burst ready every transfer interval.

This does not reduce the latency of one request. It overlaps the latencies of many requests.

3.5 Coalescing vs. Latency Hiding

These solve different problems:

Coalescing → make each warp memory request efficient

Latency hiding → keep enough independent work in flight while requests wait

A kernel can have perfectly coalesced accesses and still achieve low bandwidth if it does not expose enough outstanding work. Conversely, a kernel can have high occupancy and many requests while wasting bandwidth through poor coalescing.

4 Thread Coarsening

4.1 Motivation

Fine-grained parallelism is not free. If neighboring blocks repeatedly load the same data or repeat similar control/synchronization work, the decomposition may be too fine.

Thread coarsening assigns multiple units of work to one thread or one block:

less exposed parallelism in exchange for more reuse and less redundant overhead

4.2 Matrix Multiplication Model

Let

$$M \in \mathbb{R}^{H \times K}, \quad N \in \mathbb{R}^{K \times W}, \quad P = MN \in \mathbb{R}^{H \times W}.$$

Each output element is

$$P_{ij} = \sum_{k=0}^{K-1} M_{ik} N_{kj}.$$

Let

$$T = \text{tile width}, \quad C = \text{coarsening factor}.$$

In the baseline tiled kernel,

$$1 \text{ block} \rightarrow 1 \text{ output tile}$$

whereas in the coarsened kernel,

$$1 \text{ block} \rightarrow C \text{ horizontally adjacent output tiles}.$$

A thread with local coordinates (t_y, t_x) computes the same local position in all C output tiles.

4.3 Index Mapping

For block coordinates (b_y, b_x) ,

$$\text{row} = b_y T + t_y.$$

The first output column owned by the thread is

$$\text{colStart} = b_x C T + t_x.$$

The c -th coarse output is therefore

$$\text{col}(c) = \text{colStart} + cT$$

for

$$c = 0, 1, \dots, C - 1.$$

The thread maintains C independent accumulators:

$$Pvalue[c] \leftrightarrow P_{row, colStart+cT}$$

4.4 Memory Mapping

The full matrices belong in global memory:

$$M, N, P \rightarrow \text{global memory.}$$

The tiles reused by all threads in a block belong in shared memory:

$$M^{(b_y, p)}, N^{(p, b_x C + c)} \rightarrow \text{shared memory.}$$

The thread-private partial sums belong in registers:

$$Pvalue[0], \dots, Pvalue[C-1] \rightarrow \text{registers.}$$

For a fixed phase p , the M tile is loaded once and reused across all c . Only one N tile needs to exist in shared memory at a time.

4.5 Execution Timeline

For each phase:

$$\text{load } M \text{ tile} \rightarrow \left[\begin{array}{c} \text{load one } N \text{ tile} \\ \text{synchronize} \\ \text{compute} \\ \text{synchronize before overwrite} \end{array} \right]_{c=0}^{C-1}$$

The first synchronization prevents threads from reading incomplete shared-memory data. The second prevents some threads from overwriting a shared tile while other threads are still reading it.

4.6 Core Kernel

```
constexpr int TILE_WIDTH = 16;
constexpr int COARSE_FACTOR = 4;

__global__ void coarse_matmul(
    const float* M,
    const float* N,
    float* P,
    int H,
    int K,
    int W)
{
    int row =
```

```

    blockIdx.y * TILE_WIDTH
    + threadIdx.y;

int colStart =
    blockIdx.x
    * COARSE_FACTOR
    * TILE_WIDTH
    + threadIdx.x;

__shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
__shared__ float Nds[TILE_WIDTH][TILE_WIDTH];

float Pvalue[COARSE_FACTOR] = {0.0f};

int numPhases =
    (K + TILE_WIDTH - 1) / TILE_WIDTH;

for (int p = 0; p < numPhases; ++p) {

    if (row < H &&
        p * TILE_WIDTH + threadIdx.x < K)
    {
        Mds[threadIdx.y][threadIdx.x] =
            M[row * K
              + p * TILE_WIDTH
              + threadIdx.x];
    }
    else {
        Mds[threadIdx.y][threadIdx.x] = 0.0f;
    }

    for (int c = 0; c < COARSE_FACTOR; ++c) {

        int nRow =
            p * TILE_WIDTH + threadIdx.y;

        int nCol =
            colStart + c * TILE_WIDTH;

        if (nRow < K && nCol < W) {
            Nds[threadIdx.y][threadIdx.x] =
                N[nRow * W + nCol];
        }
        else {
            Nds[threadIdx.y][threadIdx.x] = 0.0f;
        }

        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; ++k) {
            Pvalue[c] +=
                Mds[threadIdx.y][k]
                * Nds[k][threadIdx.x];
        }

        __syncthreads();
    }
}

```

```

for (int c = 0; c < COARSE_FACTOR; ++c) {
    int col =
        colStart + c * TILE_WIDTH;

    if (row < H && col < W) {
        P[row * W + col] = Pvalue[c];
    }
}
}

```

4.7 Launch Configuration

A block still contains

$$T \times T$$

threads, but it now covers

$$T \times (CT)$$

output elements.

Therefore,

$$\boxed{blockDim = (T, T)}$$

and

$$\boxed{gridDim.y = \left\lceil \frac{H}{T} \right\rceil, \quad gridDim.x = \left\lceil \frac{W}{CT} \right\rceil.}$$

A typical launch is:

```

dim3 blockDim(TILE_WIDTH, TILE_WIDTH);

dim3 gridDim(
    (W + COARSE_FACTOR * TILE_WIDTH - 1)
    / (COARSE_FACTOR * TILE_WIDTH),
    (H + TILE_WIDTH - 1)
    / TILE_WIDTH
);

coarse_matmul<<<gridDim, blockDim>>>(
    d_M, d_N, d_P, H, K, W
);

```

4.8 Performance Reasoning

Consider C adjacent output tiles in one phase.

The baseline tiled kernel loads

$$CT^2$$

elements from M and

$$CT^2$$

elements from N , for a total of

$$\boxed{2CT^2}$$

input elements.

With coarsening, the M tile is reused across all C outputs:

$$T^2 + CT^2 = \boxed{(C + 1)T^2}.$$

The traffic ratio is

$$\frac{\text{coarsened traffic}}{\text{baseline traffic}} = \frac{C + 1}{2C},$$

which approaches

$$\frac{1}{2}$$

as $C \rightarrow \infty$.

The reduction in traffic is

$$\boxed{1 - \frac{C + 1}{2C} = \frac{C - 1}{2C}}$$

so the gain saturates.

Ignoring output stores, the baseline arithmetic intensity is

$$AI_{\text{baseline}} = \frac{2CT^3}{2CT^2 \cdot 4} = \boxed{\frac{T}{4}}.$$

The coarsened version gives

$$AI_{\text{coarse}} = \frac{2CT^3}{(C + 1)T^2 \cdot 4} = \boxed{\frac{CT}{2(C + 1)}}.$$

For $T = 16$ and $C = 4$,

$$AI_{\text{baseline}} = 4 \text{ FLOP/B},$$

$$AI_{\text{coarse}} = 6.4 \text{ FLOP/B.}$$

4.9 Trade-Offs

Coarsening increases reuse, but it also increases per-thread state:

$$C \uparrow \Rightarrow \text{accumulators/thread} \uparrow \Rightarrow \text{register pressure} \uparrow .$$

A fixed register file per SM means that higher register usage per thread can reduce the number of resident blocks and warps:

$$\boxed{\text{register pressure} \uparrow \Rightarrow \text{resident warps/SM} \downarrow \Rightarrow \text{occupancy may decrease}}$$

This can weaken the latency-hiding mechanism studied earlier.

Coarsening also reduces the total number of blocks:

$$grid_x \approx \frac{W}{CT},$$

so

$$\boxed{C \uparrow \Rightarrow \text{exposed parallelism} \downarrow .}$$

This is harmless if the remaining grid is still large enough to saturate the GPU, but it can underutilize the device for small workloads.

The overall trade-off is therefore

$$\boxed{\underbrace{\text{reuse} \uparrow, \text{traffic} \downarrow, AI \uparrow}_{\text{benefit}} \quad \text{vs.} \quad \underbrace{\text{register pressure} \uparrow, \text{occupancy may} \downarrow, \text{parallelism} \downarrow}_{\text{cost}}}$$

5 Checklist of Common CUDA Optimizations

The chapter closes by organizing the major optimization strategies into a common framework.

Optimization	Main benefit	Main strategy / trade-off
Maximize occupancy	More resident work to hide pipeline and memory latency	Tune threads/block, registers/thread, and shared memory/block
Coalesced global-memory access	Less wasted global-memory traffic and better bandwidth utilization	Align warp access patterns with memory layout; use corner turning when needed
Minimize control divergence	Higher SIMD efficiency and fewer idle lanes	Map similar work to threads in the same warp; reorganize data/work distribution
Tile reused data	Reduce repeated global-memory traffic	Load once into shared memory or registers, then reuse many times
Privatization	Reduce atomic contention and serialization	Update private copies first, then merge into the global result
Thread coarsening	Reduce redundant loading, synchronization, control, or other parallel overhead	Assign multiple work units to each thread/block, trading parallelism for reuse

6 Know the Bottleneck Before Optimizing

The most important optimization principle is:

Optimize the actual bottleneck, not the code pattern that merely looks familiar.

An optimization usually consumes more of one resource to reduce pressure on another.

Examples:

shared-memory tiling : shared memory usage $\uparrow \Rightarrow$ global-memory traffic $\downarrow$

thread coarsening : registers/thread $\uparrow$, parallelism $\downarrow \Rightarrow$ redundant traffic/overhead $\downarrow$

If global-memory bandwidth is the bottleneck, tiling or coalescing may help. If occupancy is already limited by shared memory or registers, the same optimization can make performance worse.

Therefore the practical workflow is:

understand $\rightarrow$ measure $\rightarrow$ identify bottleneck $\rightarrow$ optimize $\rightarrow$ measure again

Performance is also architecture-dependent. The same kernel may be memory-bound on one GPU and compute- or occupancy-limited on another.